

SAI Happy Farms Farm ERP — Customer Handover & System Guide

Version: 1.0

Date: 04 October 2026

Status: Official Handover Document

Prepared for: Customer

Document Control

Attribute	Details
Document Version	1.0
Date	04 October 2026
Status	Final / Handover Ready
Intended Audience	Customer, Management, Authorized Office Users
Purpose	Provide a high-level system overview, architecture, and feature registry for formal project handover.
Reference Documents	SRS Version 1.0, SRS Implementation Audit Report, Security Test Results

1. Introduction

The SAI Happy Farms Farm ERP is a comprehensive digital poultry management platform designed to transition daily farm management from manual paper registers to a centralized, real-time digital system.

Purpose of the System

The system is built to eliminate mathematical and reporting errors, prevent unauthorized cross-farm data access, and provide immediate visibility into critical farm Key Performance Indicators (KPIs).

Problems Solved

- **Data Delay:** Replaces delayed, end-of-week paper reports with real-time, instantly synchronized data.
- **Accuracy:** Enforces strict numeric constraints and automatically computes conversions (e.g., feed in kilograms to grams per bird) and percentages.
- **Security:** Physically isolates farm data ensuring users can only interact with explicitly assigned farms, preventing data leakage (Broken Object Level Authorization).

- Double Entry: Atomic transactions ensure master inventory correctly deducts mortality, culling, and feed stock accurately, even during report corrections.

Who Uses the System

The ERP accommodates three operational tiers: Farmers (field data entry), Supervisors (monitoring and operational KPIs), and Administrators (overall management and analytics).

2. System Overview

The ERP architecture utilizes a modern, decoupled stack to ensure security and real-time performance.

- Frontend: React 19 Single Page Application built with Vite and TypeScript.
- Backend: Express.js (v4.18) API operating alongside the Firebase Admin SDK. The backend strictly handles privileged administrative operations (e.g., user assignment, role management).
- Database: Firebase Firestore, utilizing atomic transactions for all critical data paths.
- Authentication: Firebase Authentication providing secure Email/Password access mechanisms.
- Data Flow:
 1. Farmer submits mobile web form.
 2. Client-side input validation triggers.
 3. Firestore executes an atomic transaction against the Master Inventory.
 4. Real-time listeners push the updated data to the Analytics Engine.
 5. Supervisor/Admin dashboards re-render immediately.
- Analytics: Recharts-driven visual dashboard computing real-time live data directly from daily reports.
- Security: Role-Based Access Control (RBAC) securely locked down via robust Firestore Security Rules and backend API validation middleware.

3. User Roles

The system enforces strict permission boundaries across three distinct roles:

Farmer

- Purpose: Daily field data entry.
- Permissions: Can log in and submit or correct daily reports strictly for the farm assigned to them.

- Restrictions: Cannot view other farms, cannot access analytics dashboards, and cannot manage other users.

Supervisor

- Purpose: Operational monitoring and KPI tracking.
- Permissions: Can access the Enterprise Analytics Dashboard, view charts, and monitor real-time KPIs strictly for explicitly assigned farms.
- Restrictions: Cannot edit farm data, cannot add or remove users, and cannot view data for unassigned farms.

Admin

- Purpose: Complete system administration.
- Permissions: Global read/write access to all farms and analytics. Can provision users, reassign farms, execute imports, manage master flock/feed stock, and view 14/21/30-day production predictions.

4. Complete Feature Register

Feature	Description	User Role	Status	Verification
Authentication	Secure email/password login.	All	Implemented & Verified	Firebase Auth
Role-Based Access Control	Explicit farm and role isolation.	All	Implemented & Verified	Firestore Rules & 179 Sec Tests
Daily Data Entry	Step-by-step wizard for farm metrics.	Farmer	Implemented & Verified	Sync Tests T04, T05
Data Correction (V2)	Same-day report delta-correction.	Farmer	Implemented & Verified	Sync Test T06
KPI Calculations	Real-time computation of metrics.	Sup / Admin	Implemented & Verified	Client-side Engine
Dashboard Analytics	Visual charts (Production, Feed, etc.).	Sup / Admin	Implemented & Verified	Enterprise Dashboard
Farm Ranking	Performance score ranking table.	Sup / Admin	Implemented & Verified	SupervisorRankingsPage
Data Filtering	7-day and 30-day boundary filters.	Sup / Admin	Implemented & Verified	Sync Test T13
Production Prediction	14/21/30 day statistical	Admin	Implemented & Verified	Prediction Engine

	forecasts.			
Language Support (EN, KN, TE, TA)	Instant UI label translations.	Farmer	Implemented & Verified	i18next Configuration
Language Support (Hindi)	Hindi language translations.	Farmer	Not Implemented	Not verified from available project evidence
In-App Alerts	Visual alerts for missing submissions.	Sup / Admin	Implemented — Verification Pending	In-app visual alerts only
Push/Email Notifications	External notifications.	Sup / Admin	Not Implemented	No FCM/Email integration
Data Search & Filtering	Free text search capabilities.	Sup / Admin	Not Implemented	No free text index
Report Export (Sheets/CSV)	CSV/Excel data export.	Admin	Not Implemented	Not verified from available project evidence
Historical Data Import	Batch import via XLSX/CSV/XML.	Admin	Requirement / Requested	Not verified from available project evidence
Import Revert / Rollback	Rollback of imported batches.	Admin	Requirement / Requested	Not verified from available project evidence

5. Complete Business Workflow

1. Login: The user authenticates. The system verifies their role and retrieves their `farmId` assignments.
2. Data Entry: The farmer proceeds through a 4-step mobile-optimized wizard.
3. Validation: Immediate numeric validation verifies data ranges (e.g., negative numbers, temperature bounds).
4. Submission: The application packages the data and pushes it to Firestore.
5. Database Transaction: An atomic backend transaction checks the current master inventory, verifies limits (e.g., mortality cannot exceed live birds), deducts the stock, and records the report.
6. KPI Calculation: The client-side Analytics Engine fetches the updated reports and runs standard performance formulas.
7. Dashboard & Monitoring: Supervisors and Admins immediately see the updated metrics visually rendered on their dashboards.

6. Security & Data Protection

The system has been heavily hardened against unauthorized access:

- Authentication: Mandatory for all access.
- Role-Based Access (RBAC): UI route protection explicitly blocks users from accessing unauthorized pages.
- Firestore Security Rules: A rigorous contract mathematically rejects read/write attempts if the requesting user's UID does not possess the correct `farmId` authorization. This ensures that cross-farm data leakage is impossible at the database level.
- Error Sanitization: Unhandled backend server errors are sanitized. Internal stack traces and raw database errors are replaced with generic standard messages to prevent infrastructure leakage.
- Sensitive Log Redaction: Automated middleware scrubs passwords and security tokens from application logging.
- Password Validation: Strict enforcement requires passwords to be 8 characters or more for all user roles.
- Production Logging Restrictions: Console debug and info logs are automatically stripped from the production frontend build.

7. Data Validation & Business Rules

- Bird Count: Cannot be negative.
- Mortality: Cannot exceed the current live bird count (`INSUFFICIENT\_BIRDS`).
- Culling: Cannot be negative (`>= 0`).
- Mortality + Culling: The sum cannot exceed the current live bird count.
- Feed: Cannot be negative. Cannot exceed the farm's currently available master feed stock (`INSUFFICIENT\_FEED`).
- Temperature: Strictly bounded between `-10°C` and `60°C`. Minimum must be less than or equal to maximum.
- Egg / Body Weight: Must be positive numbers.
- Ammonia: Must be bounded between `0` and `100` ppm. *(Trigger logic for alerts > 10 ppm is requested but not verified from the available project evidence).*
- Duplicate Submissions: Submitting a duplicate identical daily report is blocked via version checking (`CORRECTION\_LIMIT\_REACHED`).
- Farm Assignment: A user explicitly blocked or lacking farm assignment receives a `FARM\_NOT\_ASSIGNED` rejection on write attempts.

8. Testing & Verification Summary

The ERP underwent strict automated and manual validation. Verified metrics from the supplied testing documentation include:

- Security Test Suites: 24/24 passed.
- Security Assertions: 179 tests passed, 0 failed.
- Data Sync & UI Scenario Tests: 17 manual data sync and constraint tests executed and passed on the production database.
- Build Verification: Production build executes cleanly without diagnostic console leakage.

9. Deployment / Production Information

- Frontend: Built as a React SPA via Vite. The `dist` directory is prepared for static hosting deployment.
- Backend: Node.js API requires deployment to a capable hosting provider. The production URL must be provided to the frontend via `VITE\_API\_URL` during the build phase.
- Database: Operational on live Firebase Firestore instances.
- Current Production Status: Validated via production builds and live database tests. Final production deployment to the client's live domains must be confirmed from external deployment records.

10. Customer Acceptance

This document, along with the User Manuals and Testing Guide, constitutes the formal handover of the SAI Happy Farms Farm ERP.

Role	Name	Signature	Date
Customer Representative			
Development Lead			

End of Document